

常会请几个“验题者”编写程序。这些验题者往往还会故意编写错误或者速度较慢的程序，以确保这些程序会得到错误的结果，或者超时。对于一道算法竞赛的题目，正确性只是测试数据的最低要求，一套优秀的测试数据还要能全面地测出选手程序在正确性和效率上的缺陷，否则对辛辛苦苦写出正确程序的选手不公平。

6.3 树和二叉树

二叉树（Binary Tree）的递归定义如下：二叉树要么为空，要么由根结点（root）、左子树（left subtree）和右子树（right subtree）组成，而左子树和右子树分别是一棵二叉树。注意，在计算机中，树一般是“倒置”的，即根在上，叶子在下。

树（tree）和二叉树类似，区别在于每个结点不一定只有两棵子树。本书就是树状结构，根结点有 12 棵子树：第 1 章、第 2 章、第 3 章、……、第 12 章，而第 1 章又有 5 棵子树：1.1、1.2、……、1.5。

不管是二叉树还是树，每个非根结点都有一个父亲（father），也称父结点。

6.3.1 二叉树的编号

例题 6-6 小球下落（Dropping Balls, UVa 679）

有一棵二叉树，最大深度为 D ，且所有叶子的深度都相同。所有结点从上到下从左到右编号为 $1, 2, 3, \dots, 2^D - 1$ 。在结点 1 处放一个小球，它会往下落。每个内结点上都有一个开关，初始全部关闭，当每次有小球落到一个开关上时，状态都会改变。当小球到达一个内结点时，如果该结点上的开关关闭，则往左走，否则往右走，直到走到叶子结点，如图 6-2 所示。

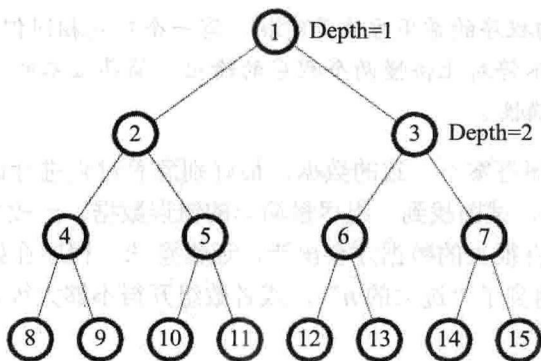


图 6-2 所有叶子深度相同的二叉树

一些小球从结点 1 处依次开始下落，最后一个小球将会落到哪里呢？输入叶子深度 D 和小球个数 I ，输出第 I 个小球最后所在的叶子编号。假设 I 不超过整棵树的叶子个数。 $D \leq 20$ 。输入最多包含 1000 组数据。

样例输入:

```
4 2
3 4
10 1
2 2
8 128
16 12345
```

样例输出:

```
12
7
512
3
255
36358
```

【分析】

不难发现, 对于一个结点 k , 其左子结点、右子结点的编号分别是 $2k$ 和 $2k+1$ 。这个结论非常重要, 请读者引起重视。

提示 6-11: 给定一棵包含 2^d 个结点 (其中 d 为树的高度) 的完全二叉树, 如果把结点从上到下从左到右编号为 $1, 2, 3, \dots$, 则结点 k 的左右子结点编号分别为 $2k$ 和 $2k+1$ 。

这样, 不难写出如下的模拟程序:

```
#include<stdio>
#include<string>
const int maxd = 20;
int s[1<<maxd]; //最大结点数为  $2^{\max d}-1$ 
int main() {
    int D, I;
    while(scanf("%d%d", &D, &I) == 2) {
        memset(s, 0, sizeof(s)); //开关
        int k, n = (1<<D)-1; //n 是最大结点编号
        for(int i = 0; i < I; i++){ //连续让 I 个小球下落
            k = 1;
            for(;;) {
                s[k] = !s[k];
                k = s[k] ? k*2 : k*2+1; //根据开关状态选择下落方向
                if(k > n) break; //已经落“出界”了
            }
        }
        printf("%d\n", k/2); //“出界”之前的叶子编号
    }
}
```

```

    return 0;
}

```

尽管在本题中，每个小球都是严格下落 $D-1$ 层，但用“if(k > n) break”的方法判断“出界”更具一般性，所以上面的代码采用了这种方法。

这个程序和前面用数组模拟盒子移动的程序有一个共同的缺点：运算量太大。由于 I 可以高达 2^D-1 ，每组测试数据下落总层数可能会高达 $2^{19} \times 19 = 9961472$ ，而一共可能有 10000 组数据……

每个小球都会落在根结点上，因此前两个小球必然是一个在左子树，一个在右子树。一般地，只需看小球编号的奇偶性，就能知道它是最终在哪棵子树中。对于那些落入根结点左子树的小球来说，只需知道该小球是第几个落在根的左子树里的，就可以知道它下一步往左还是往右了。依此类推，直到小球落到叶子上。

如果使用题目中给出的编号 I ，则当 I 是奇数时，它是往左走的第 $(I+1)/2$ 个小球；当 I 是偶数时，它是往右走的第 $I/2$ 个小球。这样，可以直接模拟最后一个小球的路线：

```

while(scanf("%d%d", &D, &I) == 2){
    int k = 1;
    for(int i = 0; i < D-1; i++){
        if(I%2) { k = k*2; I = (I+1)/2; }
        else { k = k*2+1; I /= 2; }
    }
    printf("%d\n", k);
}

```

这样，程序的运算量就与小球编号无关了，而且节省了一个巨大的 s 数组。

6.3.2 二叉树的层次遍历

例题 6-7 树的层次遍历 (Trees on the level, Duke 1993, UVa 122)

输入一棵二叉树，你的任务是按从上到下、从左到右的顺序输出各个结点的值。每个结点都按照从根结点到它的移动序列给出（L 表示左，R 表示右）。在输入中，每个结点的左括号和右括号之间没有空格，相邻结点之间用一个空格隔开。每棵树的输入用一对空括号“()”结束（这对括号本身不代表一个结点），如图 6-3 所示。

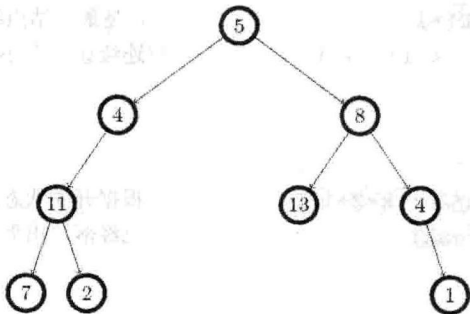


图 6-3 一棵二叉树

注意，如果从根到某个叶结点的路径上有的结点没有在输入中给出，或者给出超过一次，应当输出-1。结点个数不超过 256。

样例输入：

```
(11,LL) (7,LLL) (8,R) (5,) (4,L) (13,RL) (2,LLR) (1,RRR) (4,RR) ()
(3,L) (4,R) ()
```

样例输出：

```
5 4 8 11 13 4 7 2 1
```

```
-1
```

【分析】

受 6.3.1 节的启发，是否可以把树上的结点编号，然后把二叉树储存在数组中呢？很遗憾，这样的方法在本题中是行不通的。题目中已限制结点最多有 256 个。如果各个结点形成一条链，最后一个结点的编号将是巨大的！就算用高精度保存编号，数组也开不下。

看来，需要采用动态结构，根据需要建立新的结点，然后将其组织成一棵树。首先，编写输入部分和主程序：

```
char s[maxn]; //保存读入结点
bool read_input(){
    failed = false;
    root = newnode(); //创建根结点
    for(;;){
        if(scanf("%s", s) != 1) return false; //整个输入结束
        if(!strcmp(s, "()")) break; //读到结束标志，退出循环
        int v;
        sscanf(&s[1], "%d", &v); //读入结点的值
        addnode(v, strchr(s, ',')+1); //查找逗号，然后插入结点
    }
    return true;
}
```

程序不难理解：不停读入结点，如果在读到空括号之前文件结束，则返回 0（这样，在 main 函数里就能得知输入结束）。注意，这里两次用到了 C 语言中字符串的灵活性——可以把任意“指向字符的指针”看成是字符串，从该位置开始，直到字符“\0”。例如，若读到的结点是(11,LL)，则&s[1]所对应的字符串是“11,LL”。函数 strchr(s, ',')返回字符串 s 中从左往右第一个字符“,”的指针，因此 strchr(s, ',')+1 所对应的字符串是“LL”。这样，实际调用的是 addnode(11, "LL")。

接下来是重头戏了：二叉树的结点定义和操作。首先，需要定义一个称为 Node 的结构体，并且对应整棵二叉树的树根 root：

```
//结点类型
struct Node{
    bool have_value; //是否被赋值过
```

```
int v; //结点值
Node *left, *right;
Node():have_value(false),left(NULL),right(NULL){} //构造函数
};
```

```
Node* root; //二叉树的根结点
```

由于二叉树是递归定义的，其左右子结点类型都是“指向结点类型的指针”。换句话说，如果结点的类型为 `Node`，则左右子结点的类型都是 `Node*`。

提示 6-12：如果要定义一棵二叉树，一般是定义一个“结点”类型的 `struct`（如叫 `Node`），然后保存树根的指针（如 `Node* root`）。

每次需要一个新的 `Node` 时，都要用 `new` 运算符申请内存，并执行构造函数。下面把申请新结点的操作封装到 `newnode` 函数中：

```
Node* newnode() { return new Node(); }
```

提示 6-13：可以用 `new` 运算符申请空间并执行构造函数。如果返回值为 `NULL`，说明空间不足，申请失败。

接下来是在 `read_input` 中调用的 `addnode` 函数。它按照移动序列行走，目标不存在时调用 `newnode` 来创建新结点。

```
void addnode(int v, char* s){
    int n = strlen(s);
    Node* u = root; //从根结点开始往下走
    for(int i = 0; i < n; i++){
        if(s[i] == 'L'){
            if(u->left == NULL) u->left = newnode(); //结点不存在，建立新结点
            u = u->left; //往左走
        } else if(s[i] == 'R'){
            if(u->right == NULL) u->right = newnode();
            u = u->right;
        } //忽略其他情况，即最后那个多余的右括号
    }
    if(u->have_value) failed = true; //已经赋过值，表明输入有误
    u->v = v;
    u->have_value = true; //别忘记做标记
}
```

这样一来，输入和建树部分已经结束，接下来需要按照层次顺序遍历这棵树。此处使用一个队列来完成这个任务，初始时只有一个根结点，然后每次取出一个结点，就把它的左右子结点（如果存在）放进队列。

```
bool bfs(vector<int>& ans){
    queue<Node*> q;
```

```

ans.clear();
q.push(root); //初始时只有一个根结点
while(!q.empty()){
    Node* u = q.front(); q.pop();
    if(!u->have_value) return false; //有结点没有被赋值过,表明输入有误
    ans.push_back(u->v); //增加到输出序列尾部
    if(u->left != NULL) q.push(u->left); //把左子结点(如果有)放进队列
    if(u->right != NULL) q.push(u->right); //把右子结点(如果有)放进队列
}
return true; //输入正确
}

```

这样遍历二叉树的方法称为宽度优先遍历 (Breadth-First Search, BFS)。后面将看到, BFS 在显示图和隐式图算法中扮演着重要的角色。

提示 6-14: 可以用队列实现二叉树的层次遍历。这个方法还有一个名字, 叫做宽度优先遍历 (Breadth-First Search, BFS)。

上面的程序在功能上是正确的, 但有一个小小的技术问题: 在输入一组新数据时, 没有释放上一棵二叉树所申请的内存空间。一旦执行了 `root = newnode()`, 就再也无法访问到那些内存了, 尽管那些内存物理上仍然存在。

当然, 从技术上说, 还是可以访问到那些内存的, 如果能“猜到”那些地址。之所以说“访问不到”, 是因为丢失了指向这些内存的指针。如果读者觉得这难以理解, 想象一下丢失电话号码以后的情形: 理论上仍然可以像以前一样给朋友们打电话, 只是没有了电话簿, 查不到他们的号码了。

有一个专业术语用来描述这样的情况: 内存泄漏 (memory leak)——它意味着有些内存被白白浪费了。在实际运行的过程中, 一般很难看出这个问题: 在很多情况下, 内存空间都不会很紧张, 浪费一些空间后, 程序还是可以正常运行; 况且在整个程序结束后, 该程序占用的空间会被操作系统全部回收, 包括泄漏的那些。

提示 6-15: 如果程序动态申请内存, 请注意内存泄漏。程序执行完毕后, 操作系统会回收该程序申请的所有内存 (包括泄漏的), 所以在算法竞赛中内存泄漏往往不会造成什么影响。但是, 从专业素养的角度考虑, 请从现在开始养成好习惯, 对内存泄漏保持警惕。

下面是释放一棵二叉树的代码^①, 请在“`root = newnode()`”之前加一行“`remove_tree(root)`”:

```

void remove_tree(Node* u) {
    if(u == NULL) return; //提前判断比较稳妥
    remove_tree(u->left); //递归释放左子树的空间
    remove_tree(u->right); //递归释放右子树的空间
    delete u; //调用 u 的析构函数并释放 u 结点本身的内存
}

```

^① 这样做虽然不会出现内存泄漏, 但可能会出现内存碎片 (memory fragmentation)。

二叉树并不一定要用指针实现。接下来，把指针完全去掉。首先还是给每个结点编号，但不是按照从上到下从左到右的顺序，而是按照结点的生成顺序。用计数器 `cnt` 表示已存在的结点编号的最大值，因此 `newnode` 函数需要改成这样：

```
const int root = 1;
void newtree() { left[root] = right[root] = 0; have_value[root] = false; cnt = root; }
int newnode() { int u = ++cnt; left[u] = right[u] = 0; have_value[root] = false; return u; }
```

上面的 `newtree()` 是用来代替前面的 “`remove_tree(root)`” 和 “`root = newnode()`” 两条语句的：由于没有了动态内存的申请和释放，只需要重置结点计数器和根结点的左右子树了。

接下来，把所有的 `Node*` 类型改成 `int` 类型，然后把结点结构中的成员变量改成全局数组（例如，`u->left` 和 `u->right` 分别改成 `left[u]` 和 `right[u]`），除了 `char*` 外，整个程序就没有任何指针了。

提示 6-16：可以用数组来实现二叉树，方法是用整数表示结点编号，`left[u]` 和 `right[u]` 分别表示 `u` 的左右子结点的编号。

虽然包括笔者在内的很多选手更喜欢用数组方式实现二叉树（因为编程简单，容易调试），但仍然需要具体问题具体分析。例如，用指针直接访问比 “数组+下标” 的方式略快，因此有的选手喜欢用 “结构体+指针” 的方式处理动态数据结构，但在申请结点时仍然用这里的 “动态化静态” 的思想，把 `newnode` 函数写成：

```
Node* newnode() { Node* u = &node[++cnt]; u->left = u->right = NULL; u->have_value = false; return u; }
```

其中，`node` 是静态申请的结构体数组。这样写的坏处在于 “释放内存” 很不方便（想一想，为什么）。如果反复执行新建结点和删除结点，`cnt` 会一直增加，但是用完的内存却无法重用。在大多数算法竞赛题目中，这并不会引起问题，但也有一些对内存要求极高的题目，对内存的一点浪费就会引起 “内存溢出” 错误。常见的解决方案是写一个简单的内存池（memory pool），具体来说就是维护一个空闲列表（free list），初始时把上述 `node` 数组中所有元素的指针放到该列表中，如下所示：

```
queue<Node*> freenodes;
Node node[maxn];

void init() {
    for(int i = 0; i < maxn; i++)
        freenodes.push(&node[i]); //初始化内存池
}

Node* newnode() {
```

```

Node* u = freenodes.front();
u->left = u->right = NULL; u->have_value = false; //重新初始化该结点
freenodes.pop();
return u;
}

```

```

void deletenode(Node* u) {

```

```

    freenodes.push(u);
}

```

提示 6-17: 可以用静态数组配合空闲列表来实现一个简单的内存池。虽然在大多数算法竞赛题目中用不上,但是内存池技术在高水平竞赛以及工程实践中都极为重要。

6.3.3 二叉树的递归遍历

对于二叉树 T , 可以递归定义它的先序遍历、中序遍历和后序遍历, 如下所示:

$\text{PreOrder}(T) = T \text{ 的根结点} + \text{PreOrder}(T \text{ 的左子树}) + \text{PreOrder}(T \text{ 的右子树})$

$\text{InOrder}(T) = \text{InOrder}(T \text{ 的左子树}) + T \text{ 的根结点} + \text{InOrder}(T \text{ 的右子树})$

$\text{PostOrder}(T) = \text{PostOrder}(T \text{ 的左子树}) + \text{PostOrder}(T \text{ 的右子树}) + T \text{ 的根结点}$

其中, 加号表示字符串连接运算。例如, 对于如图 6-4 所示的二叉树, 先序遍历为 DBACEGF, 中序遍历为 ABCDEFG。

这 3 种遍历都属于递归遍历, 或者说深度优先遍历 (Depth-First Search, DFS), 因为它总是优先往深处访问。

提示 6-18: 二叉树有 3 种深度优先遍历: 先序遍历、中序遍历和后序遍历。

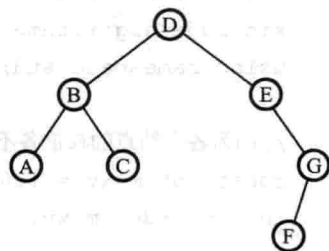


图 6-4 另一棵二叉树

例题 6-8 树 (Tree, UVa 548)

给一棵点带权 (权值各不相同, 都是小于 10000 的正整数) 的二叉树的中序和后序遍历, 找一个叶子使得它到根的路径上的权和最小。如果有多解, 该叶子本身的权应尽量小。输入中每两行表示一棵树, 其中第一行为中序遍历, 第二行为后序遍历。

样例输入:

```

3 2 1 4 5 7 6
3 1 2 5 6 7 4
7 8 11 3 5 16 12 18
8 3 11 7 16 18 12 5
255
255

```


样例输出：

```
1
3
255
```

【分析】

后序遍历的第一个字符就是根，因此只需在中序遍历中找到它，就知道左右子树的中序和后序遍历了。这样可以先把二叉树构造出来，然后再执行一次递归遍历，找到最优解。

提示 6-19：给定二叉树的中序遍历和后序遍历，可以构造出这棵二叉树。方法是根据后序遍历找到树根，然后在中序遍历中找到树根，从而找出左右子树的结点列表，然后递归构造左右子树。

代码如下：（另外，也可以在递归的同时统计最优解，不过程序稍微复杂一点，留给读者练习。）

```
#include<iostream>
#include<string>
#include<sstream>
#include<algorithm>
using namespace std;
```

//因为各个结点的权值各不相同且都是正整数，直接用权值作为结点编号

```
const int maxv = 10000 + 10;
```

```
int in_order[maxv], post_order[maxv], lch[maxv], rch[maxv];
```

```
int n;
```

```
bool read_list(int* a) {
```

```
    string line;
```

```
    if(!getline(cin, line)) return false;
```

```
    stringstream ss(line);
```

```
    n = 0;
```

```
    int x;
```

```
    while(ss >> x) a[n++] = x;
```

```
    return n > 0;
```

```
}
```

//把 in_order[L1..R1]和 post_order[L2..R2] 建成一棵二叉树，返回树根

```
int build(int L1, int R1, int L2, int R2) {
```

```
    if(L1 > R1) return 0; //空树
```

```
    int root = post_order[R2];
```

```
    int p = L1;
```

```

while(in_order[p] != root) p++;
int cnt = p-L1; //左子树的结点个数
lch[root] = build(L1, p-1, L2, L2+cnt-1);
rch[root] = build(p+1, R1, L2+cnt, R2-1);
return root;
}

int best, best_sum; //目前为止的最优解和对应的权和

void dfs(int u, int sum) {
    sum += u;
    if(!lch[u] && !rch[u]) { //叶子
        if(sum < best_sum || (sum == best_sum && u < best)) { best = u; best_sum
= sum; }
    }
    if(lch[u]) dfs(lch[u], sum);
    if(rch[u]) dfs(rch[u], sum);
}

int main() {
    while(read_list(in_order)) {
        read_list(post_order);
        build(0, n-1, 0, n-1);
        best_sum = 1000000000;
        dfs(post_order[n-1], 0);
        cout << best << "\n";
    }
    return 0;
}

```

例题 6-9 天平 (Not so Mobile, UVa 839)

输入一个树状天平，根据力矩相等原则判断是否平衡。如图 6-5 所示，所谓力矩相等，就是 $W_l D_l = W_r D_r$ ，其中 W_l 和 W_r 分别为左右两边砝码的重量， D 为距离。

采用递归（先序）方式输入：每个天平的格式为 W_l, D_l, W_r, D_r ，当 W_l 或 W_r 为 0 时，表示该“砝码”实际是一个子天平，接下来会描述这个子天平。当 $W_l = W_r = 0$ 时，会先描述左子天平，然后是右子天平。

样例输入：

1

0 2 0 4

0 3 0 1

```
1 1 1 1
2 4 4 2
1 6 3 2
```

其正确输出为 YES，对应图 6-6。

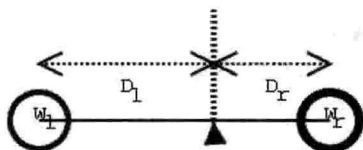


图 6-5 天平

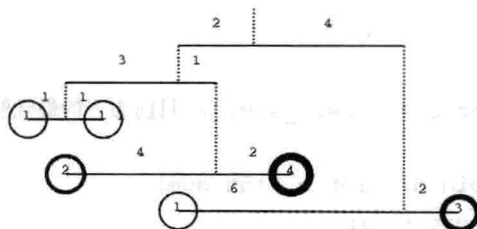


图 6-6 正确输出

【分析】

在解决这道题目之前，请先弄清楚题目的意思，尤其建议读者把样例输入画出来，以确保正确理解输入格式。

提示 6-20: 当题目比较复杂时，建议先手算样例或者至少把样例的图示画出来，以免误解题意。

这道题目的输入就采取了递归方式定义，因此编写一个递归过程进行输入比较自然。事实上，在输入过程中就能完成判断。由于使用引用传值，代码非常精简。

本题极为重要，请读者在继续阅读之前确保完全理解了下面的程序。

```
#include<iostream>
using namespace std;
//输入一个子天平，返回子天平是否平衡，参数 w 修改为子天平的总重量
bool solve(int& W) {
    int W1, D1, W2, D2;
    bool b1 = true, b2 = true;
    cin >> W1 >> D1 >> W2 >> D2;
    if(!W1) b1 = solve(W1);
    if(!W2) b2 = solve(W2);
    W = W1 + W2;
    return b1 && b2 && (W1 * D1 == W2 * D2);
}

int main() {
    int T, W;
    cin >> T;
    while(T--) {
        if(solve(W)) cout << "YES\n"; else cout << "NO\n";
        if(T) cout << "\n";
    }
}
```

```

    }
    return 0;
}

```

例题 6-10 下落的树叶 (The Falling Leaves, UVa 699)

给一棵二叉树，每个结点都有一个水平位置：左子结点在它左边 1 个单位，右子结点在右边 1 个单位。从左向右输出每个水平位置的所有结点的权值之和。如图 6-7 所示，从左到右的 3 个位置的权值和分别为 7，11，3。按照递归（先序）方式输入，用 -1 表示空树。

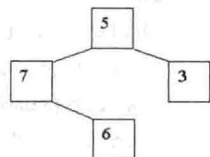


图 6-7 结点权值

样例输入：

```

5 7 -1 6 -1 -1 3 -1 -1
8 2 9 -1 -1 6 5 -1 -1 12 -1 -1 3 7 -1 -1 -1
-1

```

样例输出：

Case 1:

7 11 3

Case 2:

9 7 21 15

【分析】

本题和例题 6-9 很相似，但是实现细节比例题 6-9 略多，读者可以参考代码（这是一个不错的阅读练习）。为了节省篇幅，下面略去了唯一的全局变量 `int sum[maxn]`。

//输入并统计一棵子树，树根水平位置为 p

```

void build(int p) {
    int v; cin >> v;
    if(v == -1) return; //空树
    sum[p] += v;
    build(p - 1); build(p + 1);
}

```

//边读入边统计

```

bool init() {
    int v; cin >> v;
    if(v == -1) return false;
    memset(sum, 0, sizeof(sum));
    int pos = maxn/2; //树根的水平位置
    sum[pos] = v;
}

```

```

    build(pos - 1); build(pos + 1);
}

int main() {
    int kase = 0;
    while(init()) {
        int p = 0;
        while(sum[p] == 0) p++; //找最左边的叶子
        cout << "Case " << ++kase << ":\n" << sum[p++]; //因为要避免行末多余空格
        while(sum[p] != 0) cout << " " << sum[p++];
        cout << "\n\n";
    }
    return 0;
}

```

6.3.4 非二叉树

例题 6-11 四分树 (Quadrees, UVa 297)

如图 6-8 所示, 可以用四分树来表示一个黑白图像, 方法是用根结点表示整幅图像, 然后把行列各分成两等分, 按照图中的方式编号, 从左到右对应 4 个子结点。如果某子结点对应的区域全黑或者全白, 则直接用一个黑结点或者白结点表示; 如果既有黑又有白, 则用一个灰结点表示, 并且为这个区域递归建树。

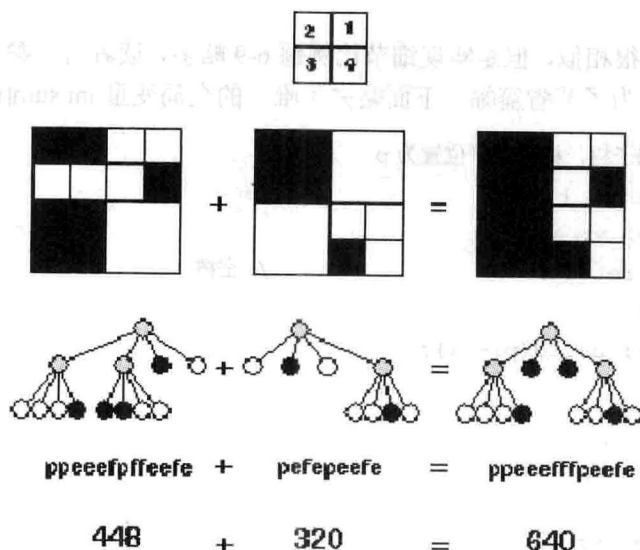


图 6-8 四分树

给出两棵四分树的先序遍历, 求二者合并之后 (黑色部分合并) 黑色像素的个数。p 表示中间结点, f 表示黑色 (full), e 表示白色 (empty)。

样例输入:

```
3
ppeeefpffeefe
pefepeeefe
peeef
peeefe
peeef
peepefefe
```

样例输出:

```
There are 640 black pixels.
There are 512 black pixels.
There are 384 black pixels.
```

【分析】

由于四分树比较特殊,只需给出先序遍历就能确定整棵树(想一想,为什么)。只需要编写一个“画出来”的过程,边画边统计即可。

```
#include<cstdio>
#include<cstring>
const int len = 32;
const int maxn = 1024 + 10;
char s[maxn];
int buf[len][len], cnt;

//把字符串 s[p..] 导出到以 (r,c) 为左上角,边长为 w 的缓冲区中
//2 1
//3 4
void draw(const char* s, int& p, int r, int c, int w) {
    char ch = s[p++];
    if(ch == 'p') {
        draw(s, p, r, c+w/2, w/2); //1
        draw(s, p, r, c, w/2); //2
        draw(s, p, r+w/2, c, w/2); //3
        draw(s, p, r+w/2, c+w/2, w/2); //4
    } else if(ch == 'f') { //画黑像素(白像素不画)
        for(int i = r; i < r+w; i++)
            for(int j = c; j < c+w; j++)
                if(buf[i][j] == 0) { buf[i][j] = 1; cnt++; }
    }
}
```